

Deep Learning: Credit Risk and Waves of AI

DENIS BURAKOV

‘Alan Turing: “This is only a foretaste of what is to come, and only the shadow of what is going to be. We have to have some experience with the machine before we really know its capabilities. It may take years before we settle down to the new possibilities, but I do not see why it should not enter any one of the fields normally covered by the human intellect, and eventually compete on equal terms. I do not think you can even draw the line about sonnets, though the comparison is perhaps a little bit unfair because a sonnet written by a machine will be better appreciated by another machine.”

— “The Mechanical Brain. Answer Found to 300 Year Old Problem.”

The Times, 11 June 1949 [28].

Today, we are witnessing extraordinary developments in technology. After decades of unfulfilled promises, autonomous agentic systems are making headway. The idea of machines performing human tasks dates back to early artificial intelligence (AI) theorists, including I.J. Good—whom we met in previous chapters and whose work extended beyond Bayesian inference to advising Stanley Kubrick along with Marvin Minsky on *2001: A Space Odyssey*. Good envisioned intelligent machines “that can far surpass all the intellectual activities of any man however clever” [6]¹.

¹Good also hypothesized that “Such machines will be feared and respected, and perhaps even loved”—a prediction that resonates strongly today as we see a growing public and commercial interest in generative AI systems.

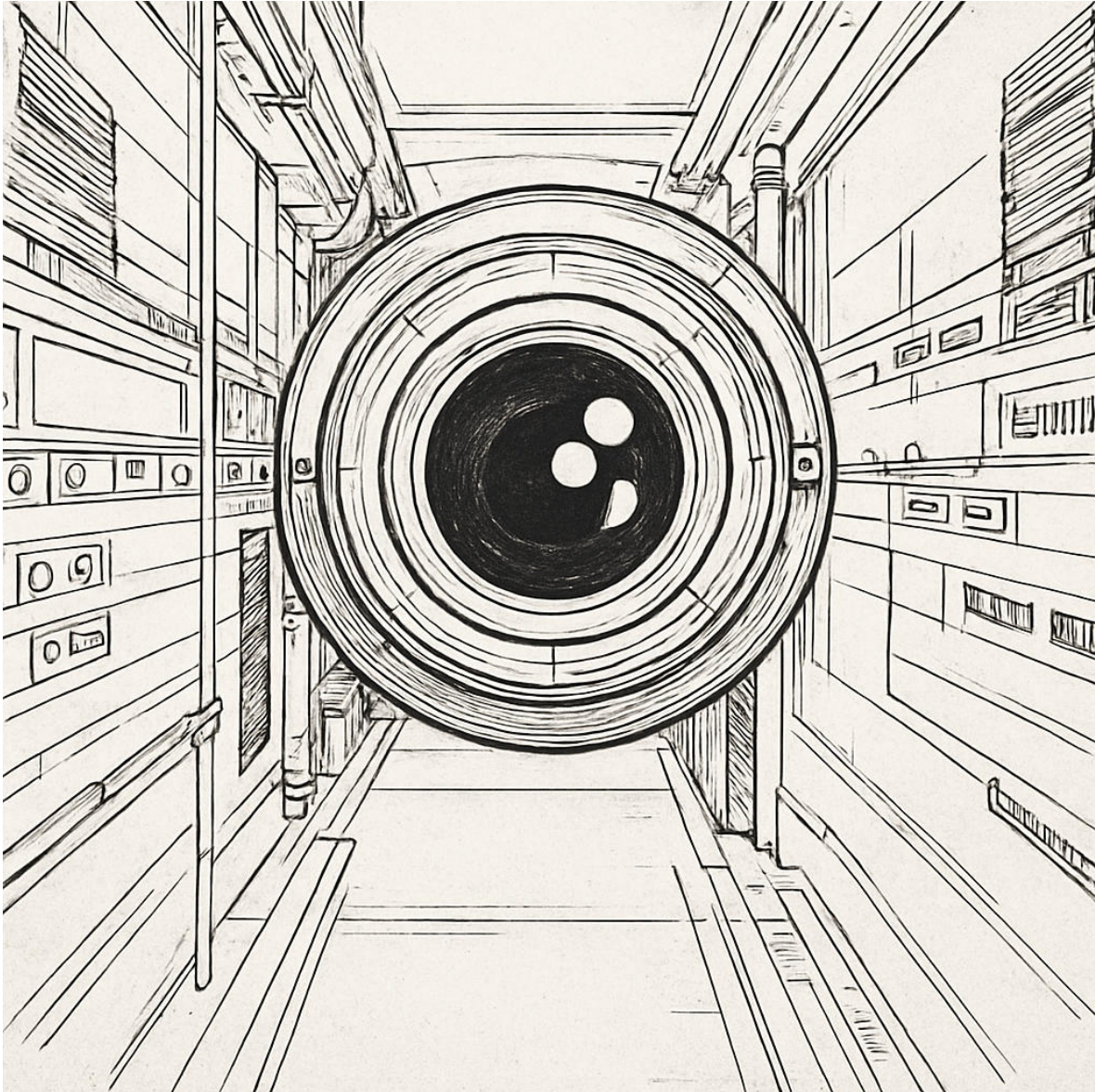


Figure 3.1: HAL 9000 from Stanley Kubrick’s *2001: A Space Odyssey* (1968). The AI’s infamous line, “I’m sorry, Dave. I’m afraid I can’t do that,” became emblematic of fears about AI acting against human interests.

The field of AI has experienced dramatic cycles of boom and bust. There were moments when the term became so damaged that researchers avoided using it entirely. As with all groundbreaking technologies, not every promise was fulfilled, leading to public disillusionment and the infamous “AI winters.” This time—in what we might call the third wave of AI (also known as agentic AI) driven by deep learning and transformer architectures—things are shaping up differently. And while there are signs of yet another bubble, with echoes of the dot-com era, the underlying technology is real and supports a wide array of practical applications.

Generative AI is transforming how we work, conduct research, and make decisions. The neural networks powering transformers and other deep learning models create what resembles a computer “brain.” Whether it truly *thinks* remains a question for philosophers, but for pragmatic

technologists in the field of financial services, the innovation is staggering and, like electricity before it, has the potential to transform how financial transactions are conducted, shaping the next generation of credit decisioning.

What should the reader expect from this chapter?

Unlike the previous chapter's linear progression through logistic regression's development, this chapter weaves together multiple historical threads to tell a more complex story, which may be particularly interesting to technologists.

Our goal is neither to provide a historical timeline of AI's development^a nor an overview of the mathematics underpinning modern AI models. Instead, we trace how credit risk modeling evolved on a parallel track to mainstream AI. Throughout this chapter, we will talk extensively about machines—be they hypothetical ones like Turing's unorganised machines or practical devices like Fair and Isaac's scorecards.

We begin this chapter with the forgotten origins of neural networks in the 1940s and 1950s, tracing their roots to Alan M. Turing's work on computability theory and revealing how his ideas about learning machines preceded the famous **Perceptron**. The chapter then explores the "AI spring" with the introduction of backpropagation for training multi-layer perceptrons (MLPs), which form the foundation of today's AI systems.

We also provide mathematical details on simple neural networks and introduce several use cases for AI in credit risk and financial services more broadly.

^aFor a concise and widely used introduction, see Chapter 1 of *Deep Learning* by Goodfellow, Bengio, and Courville [7]. Readers interested in earlier foundational work may explore Ivakhnenko's 1960s deep architectures (often considered the first modern multilayer networks) and Schmidhuber's historical surveys, which document the evolution of deep learning in greater detail. For a more technical introduction to neural networks, I recommend [25] and [22].

3.1 The Forgotten Origins of Neural Networks

The advances in computability theory by Alan Turing, Alonzo Church, and John von Neumann laid the groundwork for researchers to develop computational models of neural processes and cognition.

The history of neural networks officially starts with Warren McCulloch and Walter Pitts, whose 1943 paper introduced the first mathematical model of artificial neurons [13]. Building directly on Turing's work² on "Computable Numbers" [27], they established a basic framework: simple computational units that could be connected together to perform logical operations. Each artificial neuron would sum its inputs and fire (output 1) or remain silent (output 0) based on whether the sum exceeded a threshold—a binary decision rule that could implement logical operations like AND ($\wedge$), OR ($\vee$), and NOT ($\neg$).

Yet John von Neumann recognized a paradox in this achievement. "After the great positive contribution of Turing-cum-Pitts-and-McCulloch is assimilated," he observed, "the situation is rather worse than better than before. Indeed these authors have demonstrated in absolute and hopeless generality that anything and everything ... can be done by an appropriate mechanism,

²The conceptual connections between McCulloch-Pitts neurons and Turing's theory of computation were later articulated by John von Neumann [16], who helped bridge neuroscience, logic, and the emerging field of computer science.

and specifically by a neural mechanism—and that even one, definite mechanism can be ‘universal’” [5]. Universal computability was profound in theory, but it offered no practical guidance: *how* should one actually configure these mechanisms to perform useful tasks?

Five years after McCulloch and Pitts, in a little-known report entitled “Intelligent Machinery,” Alan Turing proposed an answer [26]. Rather than hand-wiring neural circuits for specific computations, what if the network could *learn*, randomly?

Turing called his creation “unorganised machines,” specifically *B-type unorganised machines*. While McCulloch and Pitts had shown how to wire neurons together to compute specific logical functions, Turing asked a different question: what if neurons were connected *randomly*, and the network learned through modification and training? As Copeland and Proudfoot [3] observe, Turing was probably the first to consider building computing machines from simple, neuron-like elements connected in largely random networks, then training them through experience.

Turing’s paper demonstrated that such unorganised machines could be conditioned to learn—essentially inventing the concept of trainable neural networks [28]. His ideas went beyond the static logical circuits of McCulloch and Pitts to explore how networks could adapt and improve, a conceptual advance that anticipated modern deep learning by decades.

3.2 Perceptron

Yet this work remained largely unknown. When Frank Rosenblatt, a psychologist working at the Cornell Aeronautical Laboratory, developed the Perceptron in 1958—coining the term “connectionist” and building the first hardware implementation of a learning neural network—he appears to have been unaware of Turing’s earlier exploration [21]. The parallel is striking: just as Turing had written his doctoral thesis on Gaussian errors without knowing the topic had been addressed by Jarl Waldemar Lindeberg a decade earlier, Rosenblatt independently rediscovered ideas that Turing had sketched ten years prior [3].

The Perceptron can be understood as a neural network consisting of a single artificial neuron with a threshold activation function and a single layer of weights (analogous to coefficients in a regression model). It processes Boolean-encoded input patterns (recall the one-hot encoding we covered in Chapter 2) and adjusts its weights through a learning rule. Rosenblatt’s learning algorithm drew inspiration from Donald Hebb’s 1949 work on synaptic plasticity [8], which proposed that connections between neurons strengthen when they fire together: the famous neuroscience principle “cells that fire together, wire together.” This Hebbian learning rule provided a biological foundation for how artificial networks might adapt through experience.

A reader may notice a striking resemblance between the Perceptron and logistic regression. Both compute a weighted sum of inputs and make predictions based on whether this sum exceeds a threshold. The key difference is in the activation function: the Perceptron uses a step function (outputting 0 or 1), while logistic regression uses the smooth sigmoid function (outputting probabilities between 0 and 1). Due to this property, logistic regression can be interpreted as a neural network without any hidden layers [1].

The Perceptron computes:

$$\hat{y} = \begin{cases} 1 & \text{if } \sum_{i=1}^n w_i x_i + b > 0 \\ 0 & \text{otherwise} \end{cases} \quad (3.1)$$

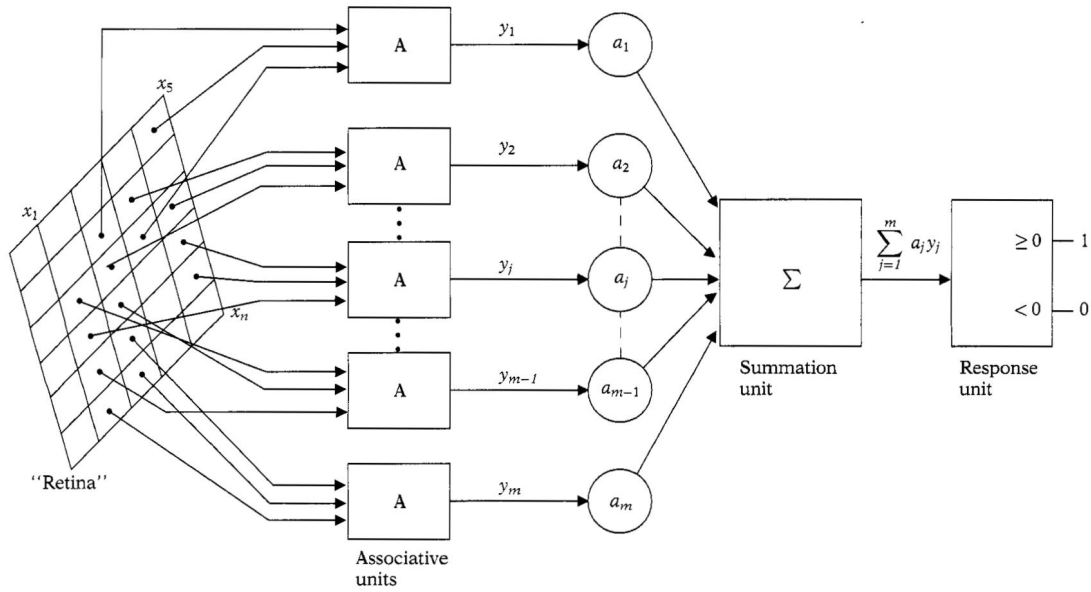


Figure 3.2: The Mark I Perceptron, developed by Frank Rosenblatt in 1958 at the Cornell Aeronautical Laboratory. This hardware implementation of a learning neural network could recognize simple patterns and was one of the first machines to demonstrate that artificial systems could learn from experience. The device used photocells to detect patterns and adjusted its weights through a process of trial and error, embodying Rosenblatt’s vision of “connectionist” computing. Image from [11].

Compare this to logistic regression: $\hat{y} = \sigma(\sum_{i=1}^n w_i x_i + b)$, where $\sigma(z) = \frac{1}{1+e^{-z}}$. The weighted sum is identical; only the final transformation differs. When logistic regression is used for classification with a threshold of 0.5 (corresponding to a weighted sum of 0), it produces a linear decision boundary just like the Perceptron, though it arrives at this boundary through a different learning process.

Rosenblatt’s learning algorithm updated weights iteratively based on errors, though it predated modern gradient descent and backpropagation. The Perceptron’s fundamental limitation, its inability to learn the exclusive OR (XOR, $\oplus$) problem, was famously demonstrated by Minsky and Papert in 1969 [14].

The XOR ($\oplus$) problem requires classifying cases where exactly one of two conditions holds, but not both or neither:

High Debt	Low Income	Risky?
No	No	No
No	Yes	Yes
Yes	No	Yes
Yes	Yes	No

A single Perceptron cannot solve this problem, as it can only create linear decision boundaries. This limitation contributed to the first AI winter that lasted until the mid-1980s.

3.3 The First AI Winter and Credit Risk’s Quiet Revolution

The Perceptron’s limitations contributed to what became known as the first AI winter, a period of reduced funding and interest that lasted until the mid-1980s. Yet while academic AI research struggled, something remarkable happened in the practical world of finance: credit scoring became one of machine learning’s earliest commercial successes.

In 1956, the same summer that John McCarthy coined the term “artificial intelligence” at the Dartmouth workshop where researchers gathered to make machines “use language, form abstractions, and solve problems reserved for humans,” two parallel revolutions were emerging. While AI pioneers theorized general artificial intelligence, Bill Fair and Earl Isaac, each investing \$400, established Fair Isaac Corporation to develop and market their credit scoring concept [20]. Their timing was fortuitous: lending offices and credit bureaus were already collecting the structured data that would power a different kind of intelligent machine.

In 1957, Fair Isaac began work focused on what it called “the credit granting problem”—the specific challenge of account origination and risk selection in consumer finance. Just as neural network researchers would later grapple with the “credit assignment problem” (determining how much each component of a network contributed to the final output), Fair Isaac was solving its own credit assignment problem—determining how much credit to assign to each applicant based on their contribution to predicted risk. By 1958, the world’s first commercial credit scoring systems for predicting consumer default had been developed and installed.

Ranking Odds: The Scorecard Machinery

Scorecards work by placing individual cases in relationship to a record of proven outcomes [18]. The device compares information about a prospective borrower to empirical patterns found at the population level, captured from accumulated historical data. By standardizing information requirements (set questions with defined answer choices), processing these data through a problem-specific algorithm, and expressing output on a numerical scale, this technology provides managers with probabilistic information for making operating decisions.

The name itself was strategic. Fair Isaac called their tool used to calculate empirical likelihood of default a “scorecard,” presumably to make it familiar by associating it with the point systems in sports. The name highlighted the technology’s two most obvious features: the numerical score output and the physical card format (which has now gone obsolete but we can still encounter it now in the digital form).

Table 3.1 shows an actual scorecard from 1977 [23], which FICO made publicly available. Each characteristic (years on job, bank accounts, credit bureau rating) has associated point values based on how borrowers with those attributes had historically performed.

Table 3.1: The Example Bank—Scorecard, 1977

CHARACTERISTICS	ATTRIBUTES				
Years on Job	1 yr to 4.5 yrs 30	Less than 1 yr 35	4.5 to 12.4 38	12.5 or more Retired 52	
Finance Company	Any 0	None Listed 22			
Department Store	None listed 10	One 15	Two 23	Two or more 26	
Years at Address	Less than 3.5 Not Ans. 30	3.5 to 7.49 38	7.5 or more 42		
Bank Account	No Bank 10	Savings Only 20	Check Only 28	Check and Save 34	
Total Income	Less than \$600 30	\$600–699 42	\$700–1,199 47	\$1,200 or more Not Ans. 59	
Worst Bureau Rating	None 16	No Rating 5	One derog –13	Two or more –40	
No. Inquiries (6 mo.)	None or One 10	No Record 7	Two –2	Three –4	Four or more –10

To calculate a score, a credit analyst would simply look up the applicant's attributes and sum the points from the scorecard. For example, an applicant with 6 years on the job (38 points), no finance company reference (22 points), two department store accounts (23 points), and a checking account (28 points) would accumulate points toward their final score [18].

This approach mirrors the logic of evidence accumulation we discussed in Chapter 1 when examining how codebreakers cracked the Enigma: each piece of information provides incremental evidence, with points (like decibans) accumulating to reach a decision threshold. Within a credit scoring system, the algorithm calculates a probability of default, a numerically expressed estimate of the odds that a person with these characteristics would repay a loan, based on how similar borrowers had historically behaved.

Engineering for Simplicity

Yet the scorecard was not an innovation that the credit industry developed on its own. Its development required bringing together statistical theory, practical business needs, and the emerging data infrastructure of credit bureaus.

Fair Isaac’s approach succeeded precisely because it didn’t require general intelligence. The problem was well-defined: predict default using historical patterns to avoid negative outcomes in future decisions. Although Fair Isaac inherited its philosophical outlook from cybernetic theories of control and communication [20], their solution was pragmatic rather than theoretical.

Their innovation was a decision science device called a “scorecard”—designed as “an accurate odds quoter” of an applicant’s future behavior [18]. Early scorecards served as aids for determining whether credit should be granted to prospective applicants based on their responses to a series of set questions. The system was carefully engineered for simplicity: credit applicants answered questions (in person or on application forms) that could be classified in tables printed on cards, with associated point values added up to produce a “credit score.” This score represented the empirically assessed odds that a person with a particular combination of characteristics, compared against known outcomes of a lender’s population of clients, would default on a loan.

The design reflected practical constraints. The score needed to be simple because those calculating or interpreting it were unlikely to have statistical training (recall from Chapter 1 how deciban rounding made the job of Turing’s associates simpler). Fair Isaac collected data from lenders and bureaus, working with paper-based records. Data scientists—then called “analysts”—had to figure out, on the ground, how to constitute samples that could build statistical models adequately discriminating between “good” and “bad” risk.

According to statistical theory, and confirmed by Fair Isaac’s empirical tests, a scoring model’s predictive utility was intimately bound to the parameters of the data set from which it was modeled [19]. Early scorecards mechanically replicated choices previously made by lenders, but refined this replication by sorting populations into statistically differentiated groups, each assigned different odds of repayment.

The Bridge That Never Was: Neural Networks and Credit Scoring

By the 1960s, Fair Isaac’s credit scoring system was already transforming consumer lending proving that better credit decisions did not require complex methods as opposed to good data and sound statistical principles. Throughout the AI winters and summers that followed, credit risk modeling evolved steadily but conservatively, prioritizing interpretability, reproducibility, and regulatory defensibility.

This conservative evolution created a profound divergence. While AI researchers pursued ever more sophisticated architectures—from backpropagation in the 1980s to deep learning in the 2010s—credit scoring remained anchored to simpler methods. The reasons were practical: lenders needed to explain their decisions to regulators, justify adverse actions to applicants, and maintain stable models that would not fluctuate wildly with each model recalibration. Neural networks, with their opaque layers and sensitivity to training data, offered power but not the transparency credit decisions demanded.

3.4 Multilayer Networks and Backpropagation

The limitation of single-layer perceptrons exposed by Minsky and Papert [14] had a straightforward solution, though it would take nearly two decades to fully realize: add more layers (and different

activations as will be discovered later). A network with multiple layers of neurons—what we now call a **multilayer perceptron** (MLP), also known as *feedforward deep networks*—can learn non-linear decision boundaries, including XOR and far more complex patterns (Figure 3.3).

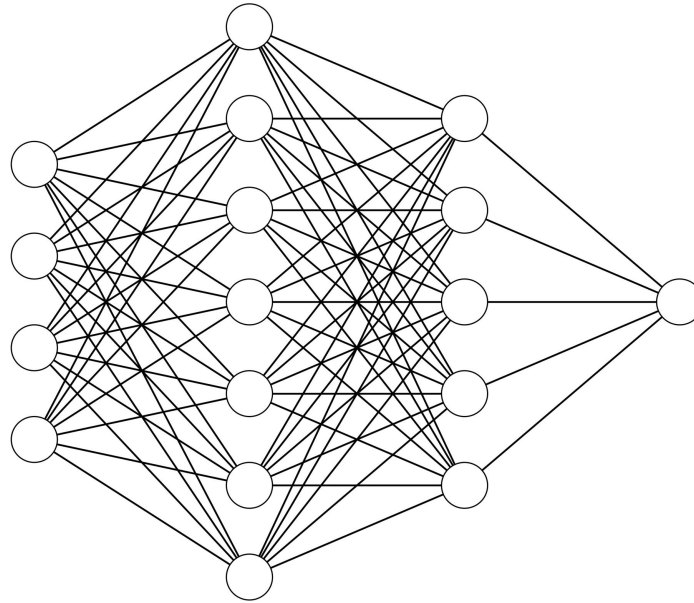


Figure 3.3: A deep neural network architecture with multiple hidden layers. Unlike the single-layer Perceptron, this network can learn complex non-linear patterns by composing multiple layers of weighted connections. Each layer transforms its inputs, allowing the network to build increasingly abstract representations from input to output.

The theoretical foundation for this had existed since the 1960s, but a practical obstacle remained: how to efficiently train these deeper networks?

Backpropagation: A Game-Changer

Before 1986, training multi-layer neural networks was largely intractable. The breakthrough came with *backpropagation*, the algorithm that would awaken neural networks from their first winter. At its core, backpropagation solves a practical problem: in a multi-layer network, how do we know which weights to adjust and by how much? The answer lies in tracing errors backward through the network, layer by layer.

Though the core idea had been independently discovered multiple times—by Paul Werbos in his 1974 PhD thesis [29], and later by David Parker [17] and Yann Le Cun [12]—it was the 1986 paper “Learning Representations by Back-Propagating Errors” by Rumelhart, Hinton, and Williams [9] that brought the algorithm to widespread attention. The impact of this work proved transformative: Geoffrey Hinton, one of the paper’s authors, received the 2024 Nobel Prize in Physics “for foundational discoveries and inventions that enable machine learning with artificial neural networks” [24].

The backpropagation algorithm’s popularity stems from being conceptually simple, computationally efficient, and because it often works [15].

Consider the network in Figure 3.4. In the **forward pass** (Step 1), we compute predictions: inputs x_1 and x_2 flow through the hidden layer (using weights $w^{(1)}$) to produce hidden activations, which then flow to the output $\hat{y}$ (using weights $w^{(2)}$). We compare $\hat{y}$ to the true value and compute a loss $\mathcal{L}$.

The **backward pass** (Step 2, backpropagation) works in reverse. Starting from the loss, we ask: how much did each weight contribute to the error?

For the output layer weights $w^{(2)}$, this is straightforward—we can see their direct impact on the prediction. But for the hidden layer weights $w^{(1)}$, the connection is indirect: they affect the hidden activations, which in turn affect the output, which affects the loss. Backpropagation uses the *chain rule* from calculus to decompose this.

The complete forward and backward passes can be expressed compactly. Note that each layer includes a bias term b (an additive constant) that allows the activation function σ (representing any differentiable nonlinearity, such as sigmoid or tanh) to shift left or right, giving the network more flexibility to fit the data:

Forward pass:

$$h = \sigma(w^{(1)} \cdot x + b^{(1)}) \quad (\text{hidden layer}) \quad (3.2)$$

$$\hat{y} = \sigma(w^{(2)} \cdot h + b^{(2)}) \quad (\text{output layer}) \quad (3.3)$$

Backward pass (gradients):

$$\frac{\partial \mathcal{L}}{\partial w^{(2)}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial w^{(2)}} \quad (\text{output layer weights}) \quad (3.4)$$

$$\frac{\partial \mathcal{L}}{\partial w^{(1)}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial h} \times \frac{\partial h}{\partial w^{(1)}} \quad (\text{hidden layer weights}) \quad (3.5)$$

Put differently, the impact of a hidden layer weight equals (how loss changes with the output) $\times$ (how output changes with hidden activations) $\times$ (how hidden activations change with that weight). By computing these gradients layer by layer in reverse, backpropagation efficiently determines how to adjust every weight in the network.

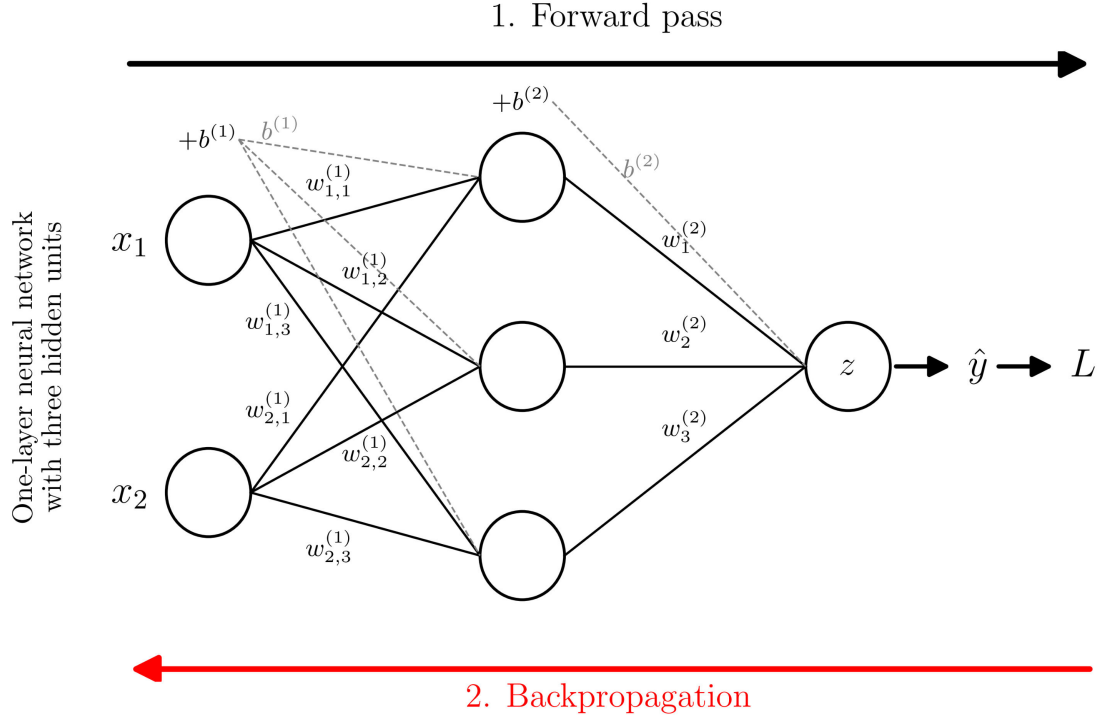


Figure 3.4: Backpropagation in a single-hidden-layer neural network. The forward pass computes predictions by propagating inputs through weighted connections. The backward pass computes gradients by propagating errors in reverse, determining how each weight contributed to the loss. The notation $w^{(1)}$ denotes weights from the input to hidden layer, while $w^{(2)}$ denotes weights from hidden to output layer. The bias terms $b^{(1)}$ and $b^{(2)}$ are additive constants applied to each layer.

The model shown in Figure 3.4 consists of:

$$\text{Inputs: } \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad (3.6)$$

$$\text{Hidden weights: } \mathbf{W}^{(1)} = \begin{bmatrix} w_{1,1}^{(1)} & w_{1,2}^{(1)} \\ w_{2,1}^{(1)} & w_{2,2}^{(1)} \\ w_{3,1}^{(1)} & w_{3,2}^{(1)} \end{bmatrix} \quad (3 \text{ neurons} \times 2 \text{ inputs}) \quad (3.7)$$

$$\text{Hidden activations: } \mathbf{h} = \sigma(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) \quad (3.8)$$

$$\text{Output weights: } \mathbf{w}^{(2)} = \begin{bmatrix} w_1^{(2)} \\ w_2^{(2)} \\ w_3^{(2)} \end{bmatrix} \quad (3.9)$$

$$\text{Output: } \hat{y} = \sigma(\mathbf{w}^{(2)T}\mathbf{h} + b^{(2)}) \quad (3.10)$$

With this model configuration (one hidden layer), we can now train a neural network to solve the XOR problem, identifying cases where exactly one condition holds (high debt or low income), but not both or neither.

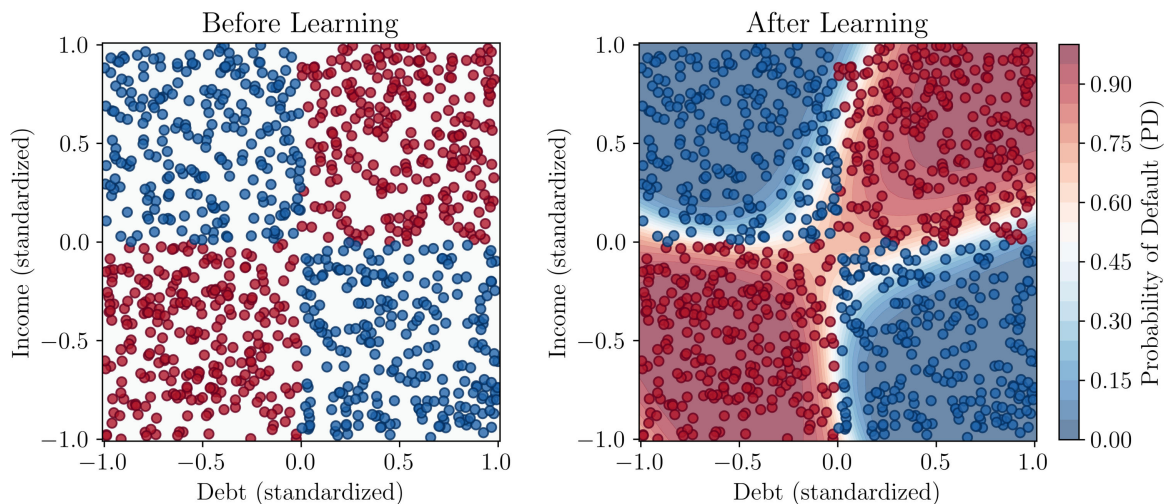


Figure 3.5: Neural network solving the XOR problem. Before training (left), the data shows an XOR pattern with red and blue classes in diagonally opposite regions. After training (right), the network learns a non-linear decision boundary (probability heatmap) that correctly classifies both classes - something impossible for single-layer models.

This case demonstrates the *universal approximation theorem* in action. The universal approximation theorem, proven independently by Cybenko [4] and Hornik [10], provides a powerful theoretical guarantee: a neural network with just a single hidden layer, the one we have in our model, can approximate any continuous function to arbitrary accuracy, given enough neurons and the right weights.

However, the theorem only guarantees that such weights *exist*. It says nothing about:

- Whether we can actually find those weights through training
- How many neurons we'd need in practice (often impractically many)
- Whether the network will generalize well to new data

This is why deeper networks, those with many layers, often work better in practice. While one hidden layer is theoretically sufficient, multiple layers can learn the same functions with fewer total neurons, discovering hierarchical patterns (like edges $\rightarrow$ features $\rightarrow$ objects) that mirror how complex problems are naturally structured.

Credit Scoring Neural Networks

Applying this model architecture to credit scoring, we train a two-layer network with 10 hidden units on 7 credit features to predict default risk. While nothing prevents us from using more hidden units or additional layers, this simple model is sufficient to illustrate the core idea.

Figure 3.6 shows what our credit neural network looks like:

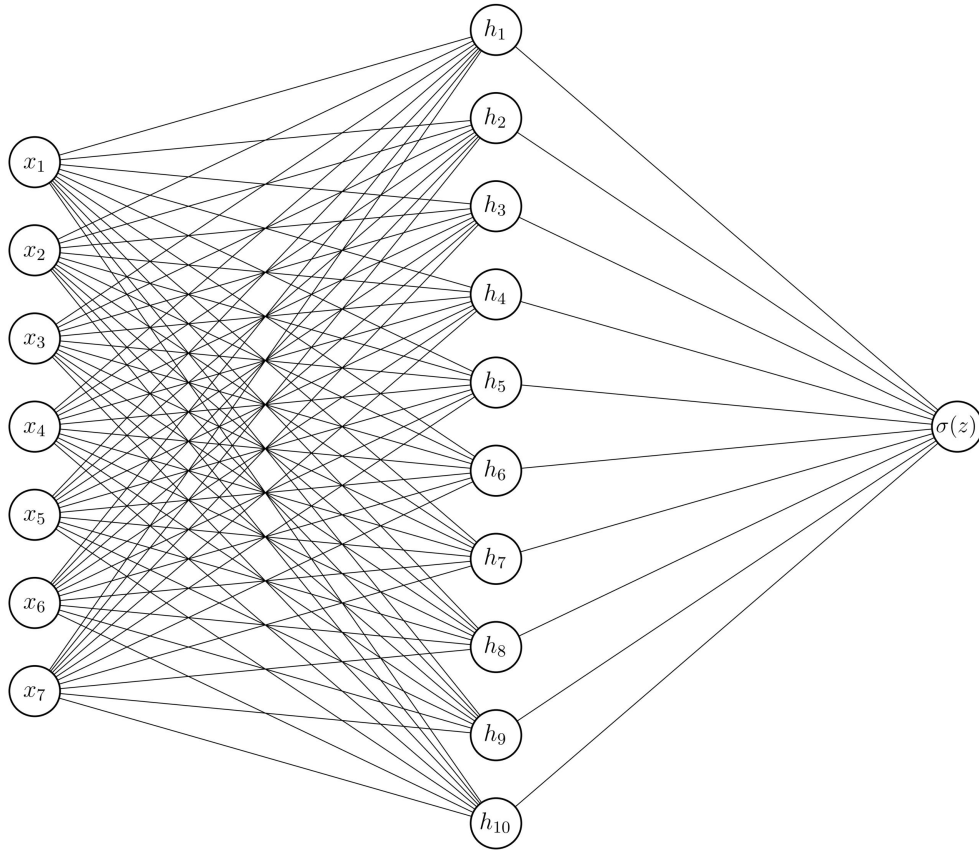


Figure 3.6: Credit scorecard MLP architecture with 7 input features, a hidden layer of 10 units with tanh activation, and a single output unit with sigmoid activation producing the probability of default estimate. The weight matrices $W^{(1)}$ and $W^{(2)}$ connect the layers, with bias terms $b^{(1)}$ and $b^{(2)}$ applied at each layer.

The learned weights (Figure 3.7) reveal interpretable patterns. In binary classification, the output weights (W_2) operate similarly to logistic regression coefficients, affecting the log-odds of default. The W_1 matrix shows how each feature activates the hidden units, while W_2 shows each unit's contribution to the final probability of default (PD).

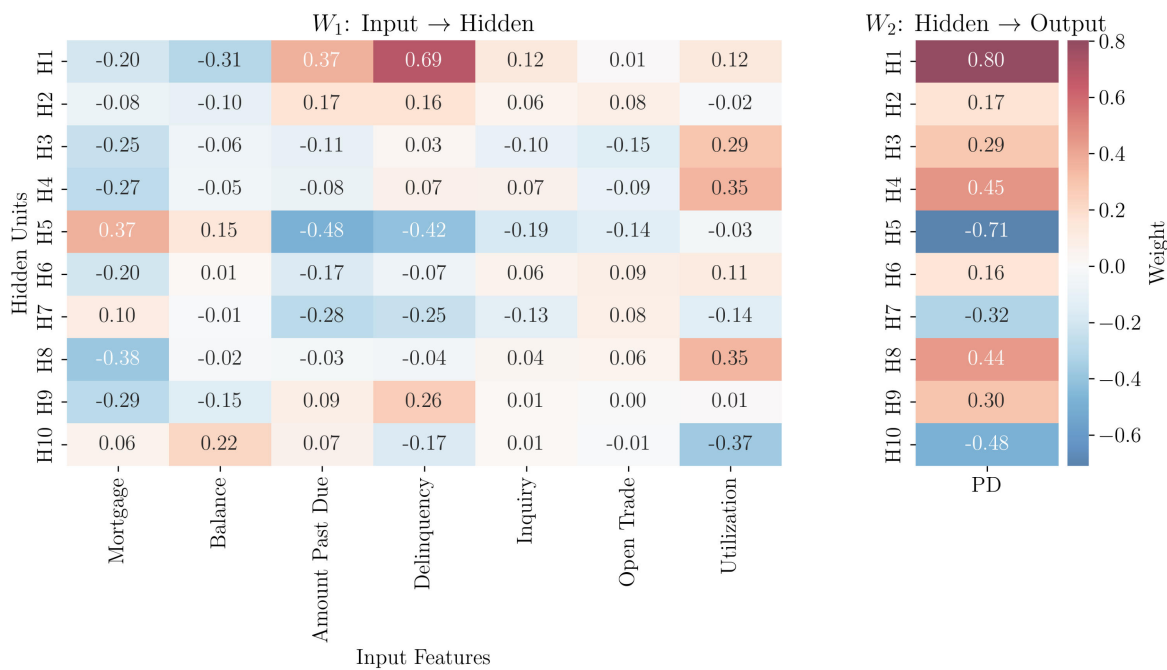


Figure 3.7: **Neural network model of credit risk.** Weight matrices learned by a two-layer neural network trained on credit risk data. The left heatmap shows W_1 weights connecting 7 input features to 10 hidden units, where red indicates positive weights (increasing hidden unit activation) and blue indicates negative weights (decreasing activation). The right heatmap shows W_2 weights connecting hidden units to the output, determining each unit’s contribution to the final default probability prediction. The improved spacing between heatmaps enhances visual clarity while maintaining the connection between the two weight layers.

In Figure 3.7, the input features are multiplied by the W_1 matrix and transformed by the hidden-layer activation function (tanh in our implementation). These hidden activations are then combined using the W_2 weights to produce the final prediction. In effect, the output neuron performs a logistic regression on these hidden activations: each hidden unit contributes a single value to the output, and the corresponding entry in W_2 acts as its coefficient in the final log-odds calculation.

Examining the most influential hidden units:

- **H1 (output weight: 0.80):** Strongly activated by delinquency (0.69) and past due amounts (0.37), with negative weights for mortgage (−0.20) and balance (−0.31). This unit serves as the primary default risk detector, identifying customers with payment problems and limited financial resources.
- **H5 (output weight: −0.71):** Strong negative weights for past due amounts (−0.48) and delinquency (−0.42), with positive mortgage weight (0.37). With its negative output weight, this unit acts as an inverse predictor: customers with clean payment histories *decrease* its activation, which paradoxically *increases* default probability in the model’s learned representation.

However, as the complexity of the neural network grows, interpreting feature importance through weight inspection becomes increasingly difficult. With deeper architectures and more hidden units, the relationship between input features and predictions becomes obscured by multiple layers of nonlinear transformations.

However, as the complexity of the neural network grows, interpreting feature importance through weight inspection becomes increasingly difficult. With deeper architectures and more hidden units, the relationship between input features and predictions becomes obscured by multiple layers of nonlinear transformations.

This complexity highlights a key concern of practitioners when using neural networks for consumer default modeling: interpretability. However, Albanesi and Vamossy [1] and [2] argue that deep learning’s ability to capture complex patterns delivers economically robust PD estimates that remain stable through credit cycles.

A central message of their work is that multidimensional, non-linear relationships exist between default and covariates—relationships that traditional linear scorecards cannot capture. Figure 3.8 provides empirical evidence of this complexity, showing how default rates vary across pairs of features in ways that defy simple linear explanations.

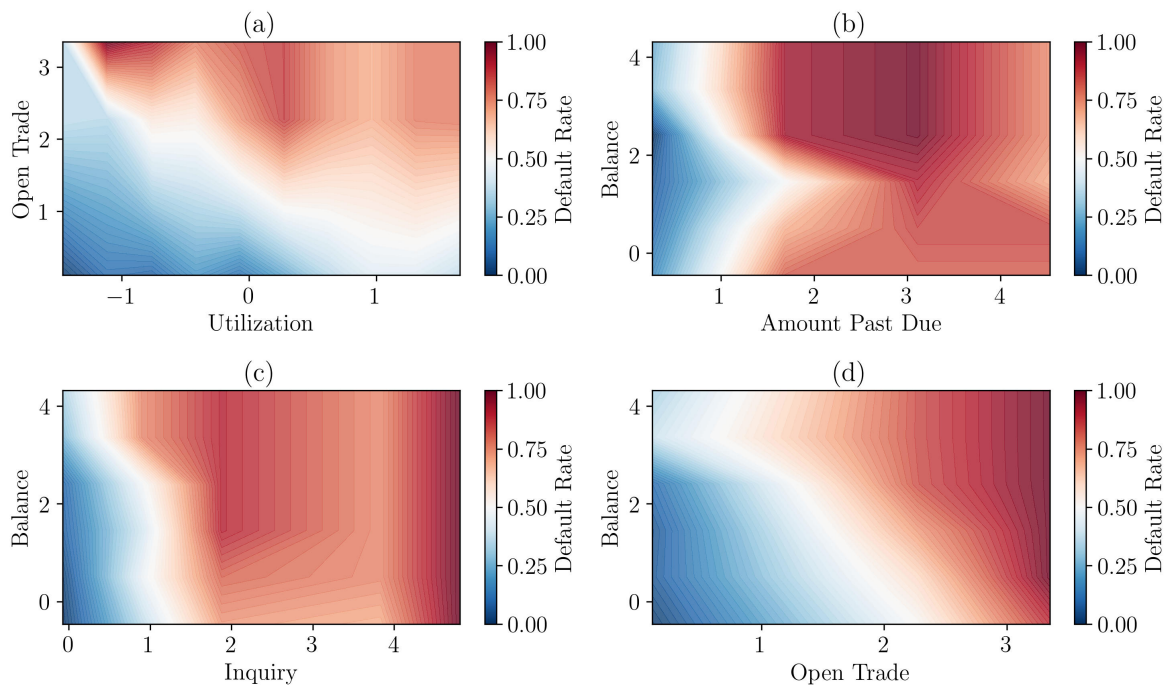


Figure 3.8: Observed default rates across feature pairs in the credit risk dataset. Each panel shows a contour plot where color intensity represents the empirical default rate (red indicates higher default risk, blue indicates lower risk).

3.5 Credit Risk in the Age of AI

The data landscape in credit risk has undergone a transformation just as profound. Digital banking and real-time payments now generate continuous streams of transaction information rather than isolated, static snapshots. Mobile applications expose spending rhythms, payment timing, and behavioral signals that traditional credit reports never captured. With the advent of

open banking, lenders can access months of granular transaction history—often with far richer temporal structure than anything previously available.

This shift from static attributes to dynamic behavioral trajectories creates an environment where deep learning is not merely an alternative to traditional scoring models, but a natural extension. Classic scorecards distill a borrower into fixed snapshots: current debt-to-income ratio, number of recent delinquencies, total credit utilization. Transaction sequences, by contrast, reveal how income fluctuates and how spending behavior adapts to the economy. Deep learning models, designed to exploit structure in sequences, can capture these temporal dynamics and cross-pattern interactions in ways that logistic regression or gradient boosting cannot.

Where tabular credit features favor tree-based methods, transaction sequences naturally align with neural networks capable of modeling order, recurrence, and long-range dependencies. This perspective forms the foundation of cash flow underwriting: assessing creditworthiness through real-time behavioral analysis rather than static historical aggregates.

Cash-Flow Underwriting

Most credit risk models rely on a point-in-time snapshot of customer information. Although these snapshots can incorporate simple trend indicators, they rarely capture the full temporal structure of a borrower's financial behavior. Scorecards, in particular, treat each variable as an independent, static attribute, leaving rich sequential patterns unmodeled.

Deep learning offers a fundamentally different approach. Many of its most influential architectures were developed precisely to handle sequences. Recurrent Neural Networks (RNNs) and their variants, such as Long Short-Term Memory networks (LSTMs), were designed to process ordered data by retaining information from previous time steps. Transformers, now the dominant architecture in sequence modeling, extend this idea using self-attention, which allows the model to weigh and relate all elements in a sequence simultaneously. This mechanism provides a flexible form of “memory” far better suited to capturing long-range dependencies than earlier methods.

The combination of rich transactional data and modern computational power makes it possible to apply these architectures directly to underwriting. What was once impractical—analyzing hundreds or thousands of transactions per customer—is now routine. Instead of relying solely on static attributes, we can model:

- trends in income and expenditures,
- fluctuations in account balances,
- variance and stability of income,
- clusters of spending behavior (e.g., saving, discretionary spending, stress spending),
- early signals of financial deterioration or resilience.

By learning from how a customer manages cash flow over time—not just from what their attributes are at a single moment—deep learning opens the door to underwriting approaches that are more dynamic, responsive, and behaviorally grounded.

Training neural networks on transactional data requires careful attention to both data preprocessing and architectural choices. Key ingredients include:

- **Feature engineering:** Extract amounts, balances, and temporal changes from raw transactions
- **Balanced sampling:** Use equal representation of defaults and non-defaults cases (50/50 split) during training phase
- **Standardization:** Scale features to zero mean and unit variance
- **Batch normalization:** Stabilize training by batch normalization
- **Sequential processing:** Structure data as time-ordered sequences at the account level to be processed in batches

The ability to collect detailed transactional histories, coupled with scalable compute, makes this shift possible. Rather than evaluating a single balance snapshot, we can analyze entire trajectories of income, expenditures, transfers, and savings patterns. From these sequences, neural networks can learn the underlying financial signatures directly: how much a customer typically spends, on what categories, how consistently they save, how volatile their cash flows are, and how they respond to periods of financial stress. In essence, transactional data allow us to reconstruct a customer's financial life in motion, since deep learning can automatically discover the features that best predict default risk.

To illustrate these ideas, consider a transformer-based model trained on real transactional data from a large lender. The model processes sequences of up to 50 transactions per customer. Instead of relying on hand-crafted features, the transformer constructs its own internal representations through multiple layers of self-attention and feed-forward layers.

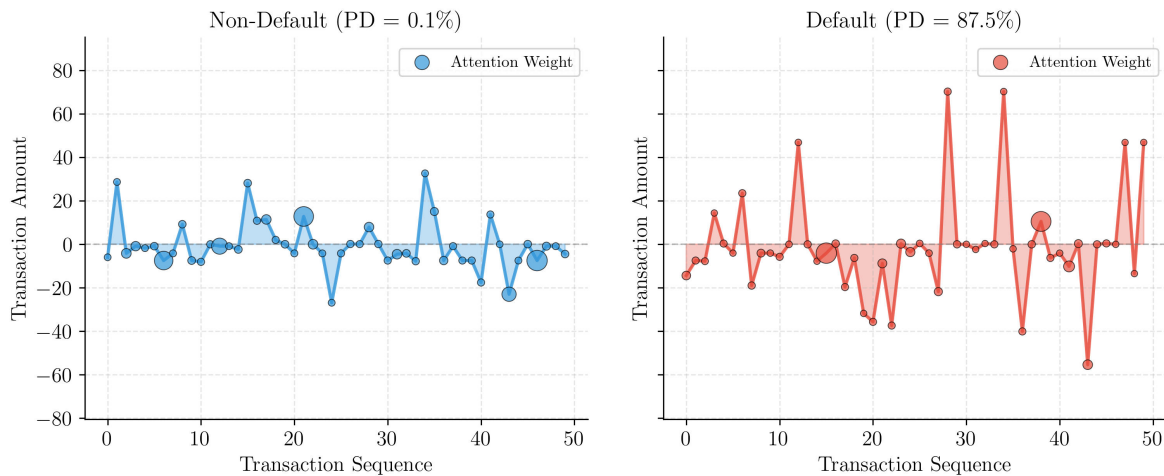


Figure 3.9: Transformer attention patterns on transaction sequences. Left: Non-default case (PD = 0.1%) shows stable balance changes with distributed attention. Right: Default case (PD = 87.5%) exhibits volatile balance changes with the transformer focusing on specific critical transactions. Marker size indicates attention weight—larger markers show which transactions the model considers most informative for prediction. The transformer automatically learns to identify temporal patterns indicative of financial stress without explicit feature engineering.

This highlights a key advantage of deep learning for credit risk: the ability to model the *shape* of financial behavior. Where a scorecard might use a single “average monthly balance” variable, a transformer considers the entire trajectory of those balances—how they rise and fall, when they spike or collapse, and how these movements relate to the timing and magnitude of individual transactions. Such temporal richness was historically inaccessible to traditional approaches but is now central to modern, behaviorally informed underwriting.

Underwriting Agents

A second frontier in the application of AI to credit risk is the automation of routine underwriting tasks. Much of underwriting consists not of subjective judgment but of a sequence of highly structured checks: retrieving bureau and bank data, validating documentation, confirming model outputs, and ensuring that a decision complies with internal policy. These steps follow clear procedural logic, making them well suited for automation through agentic AI systems.

Recent advances in large language models (LLM) make it possible to build multi-agent underwriting workflows, where each AI agent specializes in a specific part of the process. One agent may extract key financial attributes from an application or a bank statement; another may retrieve credit scores; a third may verify that a recommended action is consistent with the lending policy. These agents can communicate with one another through a form of structured message passing and present their findings to a human underwriter who owns decision-making.

This human-in-the-loop approach offers several advantages. It accelerates decision making by offloading repetitive checks, increases consistency by enforcing standardized procedures, and produces an auditable trail of intermediate reasoning steps. Importantly, these systems do not replace existing risk methodologies. Instead, they orchestrate them by retrieving the necessary data, and ensuring that all conditions for a compliant decision are met.

Viewed in this way, an “AI underwriter” is not a single monolithic model but a coordinated ensemble of specialized agents, each performing a narrow task. The analogy to human underwriting is direct: junior analysts collect documents and run checks, while senior underwriters synthesize the results and make the final decision. AI agents operate similarly, with the crucial difference that their “training” occurs through prompt-based instruction fine-tuning.

However, deploying such systems within financial services requires particular care. Most underwriting workflows cannot rely on externally hosted, general-purpose language models: doing so introduces data-privacy risks, creates dependencies on third-party infrastructure, limits reproducibility, and makes it difficult to demonstrate determinism to regulators. For these reasons, the safest and most practical approach is to deploy **local or institution-hosted open-weights LLMs**, either fine-tuned or prompt-adapted to the lender’s own credit policies, documentation standards, and operational constraints.

Within this setup, two capabilities are especially important. First, **retrieval-augmented generation (RAG)** grounds an agent’s reasoning in approved internal documents—policies, procedures, model governance reports, and historical cases—by retrieving relevant excerpts at query time. This ensures that the model bases its reasoning on verifiable institutional knowledge and behaves deterministically: given the same inputs, it will always produce the same outputs.

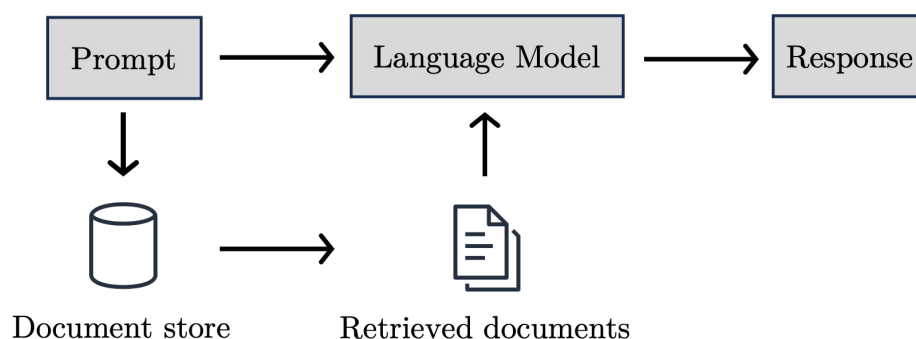


Figure 3.10: Schematic representation of the RAG process, showing the interaction between the prompt, document retrieval module, language model, and generated response.

Second, **tool-calling** allows the AI underwriter to interact with internal systems such as credit bureau interfaces, income-verification tools, scoring engines, or document repositories through controlled and auditable functions that are maintained by the lender. This provides a safe way for an LLM to trigger external actions while keeping every step transparent and reproducible.

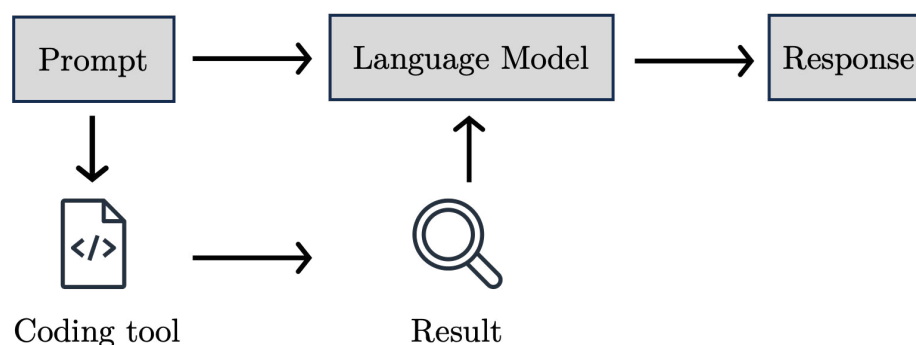


Figure 3.11: Diagram of the tool-calling process: the model interprets a user prompt, decides to invoke an external tool with structured arguments, receives the tool’s output, and generates the final response.

Together, local open-weights LLMs, RAG, and tool-calling form a controllable and auditable agentic framework for underwriting. When aligned with a lender’s internal policies and processes, these systems can automate substantial portions of the workflow while maintaining consistency, determinism, and full regulatory traceability.

Towards Credit Risk AI

This chapter has traced a remarkable arc: from Turing’s 1948 vision of “unorganised machines” that could learn through experience, through the AI winters and the rediscovery of backpropagation, to today’s transformer architectures and agentic AI systems. What began as a theoretical question—can machines learn?—has evolved into practical systems that can now assist in credit underwriting, transaction monitoring, and risk assessment.

The parallel evolution of credit scoring and neural networks is particularly noteworthy. While Fair and Isaac were building their first scorecards in the 1950s using hand-crafted features and

linear algorithms, researchers were exploring how networks of artificial neurons might learn on their own. Both approaches sought to automate human judgment, but through fundamentally different philosophies: one through careful feature engineering and interpretable odds-based risk measurements, the other through adaptive learning in high-dimensional spaces.

Today, these traditions converge. Modern credit risk systems combine the interpretability and regulatory acceptance of traditional scorecards with the pattern-recognition capabilities of deep learning. MLPs extract nonlinear relationships from transaction sequences, transformers attend to temporal patterns in payment histories, and agentic systems orchestrate these models within compliant underwriting workflows. Yet the challenge remains: how can lenders, both large and small, build on these new foundations without undermining the legacy systems that regulators understand and trust?

The third wave of AI offers unprecedented capabilities, but also unprecedented responsibilities. Turing's unorganised machines have finally arrived, but their successful deployment in credit risk will depend not just on their technical sophistication but on our wisdom in applying them. The question is no longer whether neural networks can learn credit risk patterns—they demonstrably can—but whether they can do so in ways that satisfy regulatory requirements for transparency, fairness, and stability. As we deploy these systems in financial services, we must ensure they enhance rather than obscure the decision-making process, that they reduce rather than amplify bias, and that they remain robust under the distributional shifts that inevitably occur in credit markets.

References

- [1] Stefania Albanesi and Domonkos Vamossy. *Predicting Consumer Default: A Deep Learning Approach*. NBER Working Paper 26165. National Bureau of Economic Research, 2019. URL: <https://www.nber.org/papers/w26165>.
- [2] Stefania Albanesi and Domonkos F. Vamossy. 'Credit Scores: Performance and Equity'. In: *Available at SSRN* (Aug. 2024). SSRN Working Paper. DOI: [10.2139/ssrn.4942129](https://ssrn.com/abstract=4942129). URL: <https://ssrn.com/abstract=4942129>.
- [3] B. Jack Copeland and Diane Proudfoot. 'Alan Turing's Forgotten Ideas in Computer Science'. In: *Scientific American* 280.4 (Apr. 1999), pp. 98–103. DOI: [10.1038/scientificamerican0499-98](https://doi.org/10.1038/scientificamerican0499-98).
- [4] G. Cybenko. 'Approximation by superpositions of a sigmoidal function'. In: *Mathematics of Control, Signals and Systems* 2.4 (1989), pp. 303–314.
- [5] Amanda Gefter. 'The Man Who Tried to Redeem the World with Logic'. In: *Nautilus* (Feb. 2015). Issue 21: Information. URL: <http://nautil.us/issue/21/information/the-man-who-tried-to-redeem-the-world-with-logic>.
- [6] I.J. Good. 'Speculations Concerning the First Ultra-intelligent Machine'. In: *Advances in Computers*. Vol. 6. Academic Press, 1965, pp. 31–88.
- [7] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. Cambridge, MA: MIT Press, 2016. URL: <http://www.deeplearningbook.org>.
- [8] Donald O. Hebb. *The Organization of Behavior: A Neuropsychological Theory*. New York: Wiley, 1949.

- [9] Geoffrey E. Hinton, David E. Rumelhart, and Ronald J. Williams. ‘Learning representations by back-propagating errors’. In: *Nature* 323.6088 (1986), pp. 533–536.
- [10] K. Hornik, M. Stinchcombe, and H. White. ‘Multilayer feedforward networks are universal approximators’. In: *Neural Networks* 2.5 (1989), pp. 359–366.
- [11] Laveen N. Kanal. ‘Perceptron’. In: *Encyclopedia of Computer Science*. Chichester, UK: John Wiley and Sons Ltd., Jan. 2003, pp. 1383–1385.
- [12] Yann Le Cun. ‘Une procédure d’apprentissage pour réseau à seuil asymétrique’. In: *Proceedings of Cognitiva* 85 (1985). In French, pp. 599–604.
- [13] Warren S. McCulloch and Walter Pitts. ‘A Logical Calculus of the Ideas Immanent in Nervous Activity’. In: *Bulletin of Mathematical Biophysics* 5.4 (1943), pp. 115–133.
- [14] Marvin Minsky and Seymour Papert. *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA: MIT Press, 1969.
- [15] Grégoire Montavon, Geneviève Orr, and Klaus-Robert Müller. *Neural Networks: Tricks of the Trade*. 2nd. Springer Publishing Company, Incorporated, 2012. ISBN: 364235288X.
- [16] John von Neumann. ‘The NORC and Problems in High Speed Computing’. In: *Collected Works of John von Neumann*. Ed. by A. H. Taub. Vol. 5. Originally presented in 1954. Oxford: Pergamon Press, 1961, pp. 238–247.
- [17] David B. Parker. *Learning Logic*. Tech. rep. TR-47. Massachusetts Institute of Technology, Center for Computational Research in Economics and Management Science, 1985.
- [18] Martha Poon. ‘Historicizing Consumer Credit Risk Calculation: The Fair Isaac Process of Commercial Scorecard Manufacture, 1957–circa 1980’. In: *Technological Innovation in Retail Finance: International Historical Perspectives*. Routledge, 2011, pp. 221–245.
- [19] Martha Poon. ‘Scorecards as Devices for Consumer Credit: The Case of Fair, Isaac & Company Incorporated’. In: *The Sociological Review* 55.2_suppl (2007), pp. 284–306.
- [20] Martha Ann Poon. ‘What Lenders See—A History of the Fair Isaac Scorecard’. ProQuest Dissertations & Theses, 3518969. PhD dissertation. University of California, San Diego, 2012.
- [21] Frank Rosenblatt. ‘The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain’. In: *Psychological Review* 65.6 (1958), pp. 386–408.
- [22] Simone Scardapane. *Alice’s Adventures in a Differentiable Wonderland: A Primer on Designing Neural Networks (Volume I)*. Independently published, 2024. ISBN: 9798872169895. URL: <https://www.sscardapane.it/alice-book>.
- [23] ‘The Example Bank—Scorecard’. In: *Viewpoints* I.2 (Winter 1977). Distributed by Fair Isaac; reproduced in newsletter to illustrate scorecarding practices., p. 3.
- [24] The Royal Swedish Academy of Sciences. *The Nobel Prize in Physics 2024: Press Release*. NobelPrize.org. 2024. URL: <https://www.nobelprize.org/prizes/physics/2024/press-release/> (visited on 10/08/2024).
- [25] Andrew W. Trask. *Grokking Deep Learning*. Shelter Island, NY: Manning Publications, 2019. ISBN: 9781617293702.

- [26] Alan M. Turing. *Intelligent Machinery*. Tech. rep. Reprinted in *Collected Works of A.M. Turing: Mechanical Intelligence*, edited by D.C. Ince, North-Holland, 1992. National Physical Laboratory, 1948.
- [27] Alan M. Turing. ‘On Computable Numbers, with an Application to the Entscheidungsproblem’. In: *Proceedings of the London Mathematical Society*. 2nd ser. 42 (1936). Published 1936-37, pp. 230–265.
- [28] Dermot Turing. *Prof: Alan Turing Decoded*. Stroud, GBR: The History Press, 2016.
- [29] Paul J. Werbos. ‘Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences’. PhD thesis. Cambridge, MA: Harvard University, 1974.